

Parte III: Processamento de Linguagem Natural para Linguistas

Unidade 6 - Anotações de nível de discurso

CiberExt 26-29 · FEELT38103 · Universidade Federal de Uberlândia

Agosto de 2026

Trabalhando com anotações de nível de discurso

Nas seções anteriores, exploramos diversas técnicas de Processamento de Linguagem Natural (PLN) para gerar anotações linguísticas a partir de textos puros.

Agora, o nosso foco será aprender a aproveitar anotações linguísticas que já estão prontas, dando atenção especial àquelas que analisam estruturas maiores do que uma simples oração (cláusula) — os chamados fenômenos de nível de discurso.

Ao final desta seção, você deverá ser capaz de:

- Compreender o básico do formato de anotação CoNLL-U.
- Criar um objeto *Doc* do spaCy manualmente, a partir do zero.
- Anotar trechos de texto (*Spans*) dentro de um objeto *Doc* utilizando *SpanGroups*.
- Importar corpora com anotações no formato CoNLL-U para o ecossistema do spaCy.

O formato de anotação CoNLL-U

O CoNLL-X é um formato padrão para descrever características linguísticas em uma grande variedade de idiomas (Buchholz e Marsi 2006). Ele foi criado, a princípio, para facilitar o compartilhamento de dados nas chamadas *shared tasks* (desafios computacionais abertos a pesquisadores) no campo do PLN (veja Nissim et al. 2017).

O CoNLL-U é a evolução desse formato, adaptado para suportar o modelo de Dependências Universais (*Universal Dependencies*), do qual falamos na Parte III. Hoje em dia, o CoNLL-U é amplamente usado para distribuir corpora textuais em projetos baseados no *Universal Dependencies*, mas também é o queridinho de vários outros projetos independentes de linguística computacional.

Para se ter uma ideia da sua abrangência, existem arquivos CoNLL-U não só para inúmeras línguas modernas, mas até mesmo para idiomas da antiguidade, como o Acádio (Luukko et al. 2020) e o Copta (Zeldes e Abrams 2018).

Como o CoNLL-U funciona por dentro

Os arquivos com as anotações CoNLL-U são sempre distribuídos como texto simples (texto puro, como vimos na Parte II).

Ao abrir um desses arquivos, você notará três tipos de linhas: **linhas de comentário** (comment lines), **linhas de palavra** (word lines) e **linhas em branco** (blank lines).

- As **linhas de comentário** ficam antes do texto principal e começam com o símbolo da cerquilha (#). Elas guardam informações extras (metadados) sobre o trecho de texto que vem a seguir.
- As **linhas de palavra** carregam todas as anotações referentes a uma única palavra ou pontuação (token). As frases são formadas por uma sequência contínua dessas linhas de palavra.

A estrutura dessas anotações obedece a uma organização rigorosa de colunas separadas pela tecla *Tab*, contendo 10 campos essenciais:

ID	FORM	LEMMA	UPOS	XPOS	FEATS	HEAD	DEPREL	DEPS	MISC
----	------	-------	------	------	-------	------	--------	------	------

1. ID: A posição (índice) da palavra na frase.
2. FORM: Como a palavra (ou pontuação) aparece no texto original.
3. LEMMA: A forma base ou de dicionário da palavra (lema).
4. UPOS: A categoria gramatical universal (substantivo, verbo, etc.).
5. XPOS: Uma categoria gramatical mais detalhada, específica do idioma analisado.
6. FEATS: Atributos morfológicos (gênero, número, tempo verbal, etc.).
7. HEAD: O ID da palavra que atua como o “chefe” sintático (núcleo) da palavra atual.
8. DEPREL: O tipo de relação de dependência sintática que a palavra tem com seu HEAD.
9. DEPS: Relações de dependência aprimoradas.
10. MISC: Informações adicionais de qualquer natureza (um campo coringa).

Por fim, uma **linha em branco** aparece no final das linhas de palavras para sinalizar ao sistema que a frase terminou e outra vai começar.

Trabalhando com CoNLL-U em Python

Para manipular esses arquivos de forma inteligente usando o Python, a nossa melhor amiga será a biblioteca `conllu`. Ela é leve e muito competente para ler textos no formato CoNLL-U e transformá-los em estruturas nativas do Python, fáceis de processar.

```
# Importa a biblioteca conllu
```

```
import conllu
```

Para o nosso estudo, vamos usar um arquivo CoNLL-U extraído do Georgetown University Multilayer Corpus (GUM; veja Zeldes 2017). Vamos abrir o arquivo como texto puro, lê-lo com o método `read()` e salvar o texto na variável `annotations`.

```
# Abre o arquivo de texto puro para leitura; atribui a 'data'
```

```
with open('data/GUM_whooparachute.conllu', mode="r", encoding="utf-8") as data:
```

```
# Lê o conteúdo do arquivo e atribui a 'annotations'
```

```
    annotations = data.read()
```

```
# Verifica o tipo do objeto resultante
```

```
type(annotations)
```

```
str
```

O arquivo foi carregado na memória do computador como uma gigantesca string (texto contínuo). Podemos espiar os primeiros 1000 caracteres dela para ter uma noção:

```
# Imprime os 1000 primeiros caracteres da string em 'annotations'
```

```
print(annotations[:1000])
```

```
# newdoc id = GUM_whooparachute
```

```
# sent_id = GUM_whooparachute-1
```

```
# text = How to Cope With a Double Parachute Failure
```

```
# s_type = inf
```

```
1  How how SCONJ   WRB PronType=Int    3   mark    _   Discourse=preparation:1->11
```

```
2  to to PART     TO    _   3   mark    _   _
```

```
3  Cope Cope VERB   VB   VerbForm=Inf    0   root    _   _
```

```
4  With with ADP IN    _   8   case    _   _
```

```
5  a a DET DT      Definite=Ind|PronType=Art    8   det    _   Entity=(event-1
```

```
6  Double double ADJ JJ   Degree=Pos 8   amod    _   _
```

```
7  Parachute parachute NOUN NN   Number=Sing 8   compound _   Entity=(object-2)
```

```
8  Failure failure NOUN NN   Number=Sing 3   obl    _   Entity=event-1)
```

```
# sent_id = GUM_whooparachute-2
```

```
# text = While skydiving, it is possible (yet extremely unlikely) that both your primary and
```

```
# s_type = decl
```

```
1  While while SCONJ IN    _   2   mark    _   Discourse=circumstance:2->3
```

```
2  skydiving skydiving NOUN NN   Number=Sing 6   advcl    _   Entity=(event-3)|SpaceA
```

```
3  , , PUNCT ,    _   2   punct    _   _
```

```
4  it it PRON PRP Case=Nom|Gender=Neut|Num
```

Fica bem nítida a estrutura que descrevemos acima: as linhas comentadas com a cerquilha (#), logo seguidas por colunas super organizadas com os dados da

palavra e suas análises linguísticas.

O símbolo sublinhado (_) é usado quando uma das 10 colunas não tem uma anotação definida (valor ausente).

Observe a décima coluna, a **MISC**. O corpus GUM costuma injetar campos como **Discourse** e **Entity** para marcar informações de nível superior sobre a estrutura do discurso e das entidades no texto (quem é quem na história, lugares, objetos, etc.).

A grande questão agora é: como automatizar a leitura disso no Python sem nos perdermos nessa string gigante?

Aqui entra a função `parse()` do módulo `conllu`. Ela faz exatamente o trabalho duro de “fatiar e traduzir” a string bruta em um formato legível.

```
# Usa a função parse() para analisar as anotações; armazena em 'sentences'
sentences = conllu.parse(annotations)
```

O que a função `parse()` nos devolve é uma lista do Python recheada de objetos *TokenList*. O *TokenList* é a estrutura de dados chave criada pela própria biblioteca `conllu` para abrigar orações.

Vamos verificar o que se esconde no primeiro item (`sentences[0]`).

```
sentences[0]
```

```
TokenList<How, to, Cope, With, a, Double, Parachute, Failure, metadata={newdoc id: "GUM_who
```

Maravilha, a biblioteca capturou os metadados (que estavam com #) e os disponibilizou sob a propriedade `metadata`. Vamos acessar esse dicionário de metadados diretamente:

```
# Obtém os metadados do primeiro item da lista
sentences[0].metadata

{'newdoc id': 'GUM_whoow_parachute',
 'sent_id': 'GUM_whoow_parachute-1',
 'text': 'How to Cope With a Double Parachute Failure',
 's_type': 'inf'}
```

Isso nos revela como o projeto GUM costuma estruturar a papelada: ele usa `newdoc_id` para rotular o documento inteiro, `sent_id` para rotular cada sentença isolada, `text` com a frase original limpa, e `s_type` que categoriza a sentença sob um viés gramatical (afirmação, pergunta, imperativo etc). (Zeldes & Simonson 2017: 69).

Embora o terminal retorne um formato que parece um simples dicionário, a estrutura por trás do `metadata` é um objeto especializado (chamado *Metadata*). No entanto, sua operação diária é idêntica a um dicionário nativo do Python: bastam chaves e valores.

Portanto, se precisarmos apenas do modo verbal da oração (`s_type`), usamos a sintaxe clássica de dicionário:

```
# Obtém o tipo de sentença sob a chave 's_type'
sentences[0].metadata['s_type']

'inf'
```

Isso nos retornará a string `inf`, indicando que a frase está no modo infinitivo.

E sobre as palavras individuais? Como o próprio nome aponta, o objeto *TokenList* é construído agrupando múltiplos *Tokens*.

Acessar o primeiro *Token* do nosso primeiro *TokenList* é simples:

```
# Obtém o primeiro token da primeira sentença
sentences[0][0]

{'id': 1,
 'form': 'How',
 'lemma': 'how',
 'upos': 'SCONJ',
 'xpos': 'WRB',
 'feats': {'PronType': 'Int'},
 'head': 3,
 'deprel': 'mark',
 'deps': None,
 'misc': {'Discourse': 'preparation:1->11'}}
```

Mais uma vez, recebemos as informações num arranjo que emula a estrutura de um dicionário.

Note bem a chave `misc`. O dicionário embutido nela carrega anotações cruciais sobre as “relações de discurso”. Essas relações servem para amarrar ideias através de uma teoria gramatical profunda, chamada Teoria da Estrutura Retórica (RST - Rhetorical Structure Theory) (Mann & Thompson 1988).

De forma resumida, essa anotação nos diz que existe uma relação discursiva (do tipo **preparation**) ligando as partes “1” e “11” do texto.

Porém, atenção: esses números não apontam para índices de palavras ou de sentenças soltas. Eles apontam para “Unidades Elementares do Discurso” (do inglês *Elementary Discourse Units*, ou EDU).

A segmentação em EDUs cria uma lente de aumento na qual os analistas subdividem a narrativa em fragmentos básicos e avaliam de que forma eles formam o esqueleto da história.

No molde clássico da Teoria da Estrutura Retórica (RST), as EDUs coincidem majoritariamente com cláusulas ou orações, mas não se trata de uma regra universal matemática.

Levando a análise de discurso para objetos *Doc* do spaCy

Como verificamos anteriormente, os metadados embutidos no campo `misc` (como a anotação `Discourse`) sinalizam o exato momento onde uma nova EDU **inicia**.

Se percorrermos a nossa sequência de *Tokens* rastreando o surgimento dessas anotações, seremos capazes de desenhar precisamente as fronteiras entre cada Unidade Elementar do Discurso — que muitas vezes cruzam o limite fixo das orações normais da nossa *TokenList*.

A estratégia prática é varrer todos os *Tokens* criando uma contagem progressiva e anotar qual índice abriga o valioso carimbo `Discourse`.

Vamos instanciar a variável de contagem (`counter`) e preparar o “terreno” (várias listas) para guardar com segurança as descobertas:

```
# Cria uma variável com valor 0, que usaremos para contar
# os Tokens processados
counter = 0

# Usamos estas listas para acompanhar as sentenças, as unidades de discurso
# e as relações que se mantêm entre elas.
discourse_units = []
sent_types = []
relations = []

# Cria listas vazias para a informação que extrairemos das anotações
# CoNLL-U. Essas listas serão usadas para criar um objeto Doc do
# spaCy mais adiante.
words = []
spaces = []
sent_starts = []
```

O bloco a seguir irá analisar, palavra por palavra de cada **sentence**, preenchendo as listas acima com todos os dados relevantes (marcando também onde iniciam os EDUs):

```
# Percorre cada objeto TokenList
for sentence in sentences:

    # Ao começar a percorrer uma nova sentença, define o valor da
    # variável 'is_start' como True. Isso marca o início de uma
    # sentença.
    is_start = True

    # Acrescenta o tipo da sentença atual à lista 'sent_types'
    sent_types.append(sentence.metadata['s_type'])

    # Prossegue percorrendo os Tokens do TokenList atual
```

```

for token in sentence:

    # Usa a chave 'form' para recuperar a forma da palavra do Token
    # e a acrescenta à lista 'words'.
    words.append(token['form'])

    # Verifica se este Token inicia uma sentença, avaliando se a
    # variável 'is_start' é True.
    if is_start:

        # Se o Token inicia uma sentença, acrescenta o valor True à lista
        # 'sent_starts'. Note a ausência de aspas: trata-se de um valor
        # booleano (True / False).
        sent_starts.append(True)

        # Define 'is_start' como False até que a próxima sentença comece
        # e a variável seja novamente definida como True.
        is_start = False

    # Se a variável 'is_start' for False, executa o bloco abaixo
    else:

        # Acrescenta o valor False à lista 'sent_starts'
        sent_starts.append(False)

    # Verifica se a chave 'misc' contém algo e se ela guarda o valor
    # 'Discourse'; em caso afirmativo, executa o bloco abaixo.
    if token['misc'] is not None and 'Discourse' in token['misc']:

        # A presença da chave 'Discourse' indica o início de uma nova
        # unidade elementar de discurso; acrescenta seu índice à lista
        # 'discourse_units'.
        discourse_units.append(counter)

        # Desempacota a definição da relação; começa separando o nome da
        # relação das unidades elementares de discurso. Atribui os objetos
        # resultantes a 'relation' e 'edus'.
        relation, edus = token['misc']['Discourse'].split(':')

        # Tenta dividir a anotação da relação em duas partes
        try:

            # Divide na string '->' e atribui a 'source'
            # e 'target', respectivamente.
            source, target = edus.split('->')

```

```

        # Subtrai 1 de 'source' e de 'target', porque os identificadores
        # usados no corpus GUM não começam em zero, ao contrário dos
        # nossos Spans do spaCy que correspondem às unidades elementares
        # de discurso. Também converte os números em inteiros
        # (originalmente eles são strings!).
        source, target = int(source) - 1, int(target) - 1

    # O nó raiz da árvore RST não terá alvo, o que levanta um
    # ValueError, já que só existe um item.
    except ValueError:

        # Atribui o primeiro item de 'edus' a 'source' e define
        # 'target' como None.
        source, target = edus[0], None

        # Subtrai 1 de 'source', conforme explicado acima.
        source = int(source) - 1

    # Compila a definição da relação em uma tripla e a acrescenta
    # à lista 'relations'.
    relations.append((relation, source, target))

    # Verifica se o Token atual é seguido por um espaço. Quando não é o
    # caso - por exemplo, no Token ao final de um TokenList -, essa
    # informação está disponível na chave 'misc'.
    if token['misc'] is not None and 'SpaceAfter' in token['misc']:

        # Se a chave 'misc' guarda um dicionário com a chave 'SpaceAfter'
        # com o valor 'No', executa o bloco abaixo
        if token['misc']['SpaceAfter'] == 'No':

            # Acrescenta o valor booleano False à lista 'spaces'.
            spaces.append(False)

        # Se a chave 'SpaceAfter' não for encontrada em 'misc', o token é
        # seguido por um espaço.
    else:

        # Acrescenta True à lista de espaços
        spaces.append(True)

    # Atualiza o contador ao terminar de processar um objeto Token,
    # somando 1 ao seu valor.
    counter += 1

```

O código acima é uma rotina clássica de “garimpo”. Varrer um dicionário

estruturado na unha para separar todos os grãos essenciais que, mais na frente, ajudarão a moldar um objeto *Doc* limpo, mantendo viva a bagagem de informações aprofundadas do documento original.

Como você notou, processar arquivos baseados num esquema rico (como o CoNLL-U) não admite aventureiros. Entender exatamente as engrenagens de cada modelo (aqui, a forma como o corpus GUM condiciona regras linguísticas dentro da coluna livre *misc*) exige leitura do manual, para garantir que as fatias não derretem no final do trajeto.

Na vida normal de programadores de PLN, injetar texto simples no funil do *Language* object para que o spaCy entregue, do outro lado, um arquivo rico em morfologia e marcações estruturais é rotineiro, algo que batemos o martelo na Parte II.

Entretanto, esse atalho **não serve para a missão atual**. E o porquê disso é elementar: caso deixássemos a inteligência do spaCy fragmentar a nosso texto de forma nativa e automática (ou seja, tokenizando do seu próprio jeito), correríamos o perigo real do spaCy tomar liberdades divergentes da estrutura em “pedras fundamentais” exigidas pelo CoNLL-U e nosso meticuloso rastreio cair por terra. Perderíamos a sincronia geométrica das anotações discursivas!

A via correta passa pela “fabricação artesanal” do objeto *Doc*. Carregaremos os construtores na unha a partir da módulo matricial *tokens*, somando, é claro, a bagagem intelectual (o idioma de vocabulário) presente num clássico modelo inglês simplificado (o modesto, mas competente, *en_core_web_sm*).

```
# Importa a classe Doc e a biblioteca spaCy
from spacy.tokens import Doc
import spacy

# Carrega um modelo de linguagem pequeno para o inglês; armazena em 'nlp'
nlp = spacy.load('en_core_web_sm')

Com o arcabouço pronto, injetaremos no objeto as matrizes que laboriosamente
resgatamos antes: a listagem das palavras (words), seus indícios delimitadores
interespaiais (spaces) e os batizadores de largada frasal (sent_starts).

Para ilustrar o cenário atual, as impressões a seguir trazem as 15 “primeiras
pedras” agrupadas na montagem paralela das referidas matrizes.

# Percorre os 15 primeiros itens das listas 'words', 'spaces' e 'sent_starts'.
# Usa a função zip() para buscar os itens das listas simultaneamente.
for word, space, sent_start in zip(words[:15], spaces[:15], sent_starts[:15]):

    # Imprime o item atual de cada lista
    print(word, space, sent_start)
```

```
How True True
to True False
```

```

Cope True False
With True False
a True False
Double True False
Parachute True False
Failure True False
While True True
skydiving False False
, True False
it True False
is True False
possible True False
( False False

```

Além dessas três colunas vitais, é nossa tarefa amarrar essas diretrizes ao vocabulário inerente do inglês, repassando o coração do modelo (`nlp.vocab`) por meio da propriedade `vocab` presente no `Doc`.

```

# Cria um objeto Doc do spaCy "manualmente"; atribui à variável 'doc'
doc = Doc(vocab=nlp.vocab,
          words=words,
          spaces=spaces,
          sent_starts=sent_starts
        )

```

Missão cumprida! Obtemos com sucesso a gênese de um objeto *Doc* totalmente sob nosso comando métrico e delineado pelas fronteiras autênticas do formato CoNLL-U.

```

# Recupera os Tokens até o índice 15 do objeto Doc
doc[:15]

```

How to Cope With a Double Parachute Failure While skydiving, it is possible (

Os 15 grãos textuais encadeados de forma primorosa evidenciam o encaixe, ditado perfeitamente sob batuta da listagem `words` em compasso fiel a intermitência ditada pelos sinalizadores em `spaces`.

Sem falar nos delimitadores frasais! As sentenças perfeitamente modeladas pela marcação original brilham isoladas sem a interferência natural sintática do spaCy (isso vive alojado nas dependências relativas da gaveta `sents` do novo objeto `doc`).

```

# Recupera as cinco primeiras sentenças do objeto Doc
list(doc.sents)[:5]

```

```

[How to Cope With a Double Parachute Failure,
 While skydiving, it is possible (yet extremely unlikely) that both your primary and reserve
 In the vast majority of cases, this will not occur;,
 nevertheless, in this event these coping strategies may assist.,

```

Steps]

Entretanto, por focarmos nas limitações espaciais descartando a riqueza sintática englobada na estrutura bruta CoNLL-U original (apenas absorvemos o texto e discurso), os atributos internos desses *Tokens* (como tipo da palavra) encontram-se desamparados (em estado oco).

Dúvida? Veja o deserto retornado numa tentativa de pinçar as raízes morfológicas aprofundadas no campo inerente `tag_` pertencente ao primeiro elo da frase.

```
# Obtém a etiqueta refinada de classe gramatical do Token no índice 0
doc[0].tag_
''
```

Branco nítido, inexistência provada. O dilema atual é: o spaCy condena objetos empacotados “do lado de fora” em sua linha normal de atuação do modelo `nlp`. Precisamos nós mesmos encaminhar os papéis e protocolar as operações injetando `doc` nos guichês especializados isolados (*tagger*, *parser*, etc) disponíveis pela estrada rotulada em `pipeline`! O mapa já havia sido discutido de praxe em pormenor em trecho anterior (Parte II).

Iremos transitar com as premissas em *loop* engarrafando e enriquecendo nosso arcabouço através dos guichês encadeados:

```
# Percorre os pares nome / componente disponíveis no atributo 'pipeline'
# do objeto Language 'nlp'.
for name, component in nlp.pipeline:

    # Usa uma string formatada para imprimir o 'name' do componente
    print(f"Now applying component {name} ...")

    # Fornece o objeto Doc existente ao componente e armazena as anotações
    # atualizadas na variável de mesmo nome ('doc').
    doc = component(doc)

Now applying component tok2vec ...
Now applying component tagger ...
Now applying component parser ...
Now applying component attribute_ruler ...
Now applying component lemmatizer ...
Now applying component ner ...
```

Terminado o expediente operário o objeto exhibe suas aquisições ao retornar o valor preenchido para a busca em `tag_`:

```
# Obtém a etiqueta refinada de classe gramatical do Token no índice 0
doc[0].tag_
'WRB'
```

Melhor ainda! Temos à disposição o aparato de análises finas exclusivas derivadas, evidenciadas de antemão através dos construtos sintáticos agregados em sintagmas sob o título de `noun_chunks`.

```
# Obtém os cinco primeiros sintagmas nominais do objeto Doc
list(doc.noun_chunks)[:5]
```

```
[a Double Parachute Failure,
 it,
 both your primary and reserve parachutes,
 you,
 no method]
```

O detalhe fundamental: os limites frasais pautados “na unha” (sob demanda `sent_starts`) foram preservados pelo sistema que historicamente tem a tara em atropelar e ditar suas próprias regras baseadas na árvore de sintaxe das palavras em vez da pontuação pura!

```
# Obtém as cinco primeiras sentenças do objeto Doc
list(doc.sents)[:5]
```

```
[How to Cope With a Double Parachute Failure,
 While skydiving, it is possible (yet extremely unlikely) that both your primary and reserve
 In the vast majority of cases, this will not occur;,
 nevertheless, in this event these coping strategies may assist.,
 Steps]
```

Cenário pacificado! De posse integral sob manto protetivo da anotação, caminhamos aos derradeiros ritos em infundir na raiz o viés da dimensão analítica calcada e mapeada em nível discursivo oriunda dos garimpos estipulados via `discourse_units`, atados nas nuances em `sent_types` e conexões retóricas nas amarras `relations`.

Amarrando o modo da oração às entranhas do spaCy

Começaremos carimbando o estado de humor (o tom gramatical, em suma, o modo) atado nas bases do conjunto armazenado `sent_types`.

Categorias típicas (como traçamos anteriormente) ditam a tônica em pautas relativas ao uso do tipo “infinitivo”, vertentes calcadas para “imperativo” e balizas de cunho “declarativo”.

Um mergulho rápido na listagem comprova o resgate das etiquetas.

```
# Imprime os tipos de sentença
sent_types[:10]
```

```
['inf', 'decl', 'decl', 'decl', 'frag', 'imp', 'decl', 'imp', 'imp', 'imp']
```

Nossa missão dita agora “grudar” as correspondentes chancelas na traseira das sentenças que moldam nosso modelo *Doc*. Para nos certificarmos perante

risco de incompatibilidade, atestaremos em conduta primária o nivelamento quantitativo entre o catálogo listado e os lotes dispostos encarnando o grupo frasal da arquitetura atual.

```
# Verifica o tamanho da informação de tipo de sentença e
# o número de sentenças do objeto Doc.
len(sent_types) == len(list(doc.sents))
```

True

Para injetar as peculiaridades balizadas nas regras semânticas na infraestrutura robusta do spaCy a conduta ortodoxa determina alocar “tags customizadas” (os “custom attributes”) no bojo interno associado com as particularidades dos conjuntos de texto rotulados pelas diretrizes da propriedade *Span*, que congrega, agrupa e encarna as delimitações englobando agrupamentos formados por parcelas unidas no manto de matriz **Token** na raiz interna atada e fixada de preceito pela arquitetura global *Doc*.

Ora, se pararmos e inspecionarmos pontualmente, um lote sob regência vinculada na pasta interna a nível **sents** compõe intrinsecamente um viés formatado de acordo com uma vestimenta autêntica sob carimbo **Span**.

```
# Imprime a primeira sentença do Doc e seu type()
list(doc.sents)[0], type(list(doc.sents)[0])
```

(How to Cope With a Double Parachute Failure, `spacy.tokens.span.Span`)

Através da constatação e premissa referendada, a investida que estampa nossa meta em injetar a taxonomia vinculativa aos atributos providos nas sentenças requereria e apela pela convocação pontual (uso do artifício “import”) associando as bibliotecas balizadoras para lidar com *SpanGroup* e o genérico formatado em *Span*.

```
# Importa as classes SpanGroup e Span do spaCy
from spacy.tokens.span_group import SpanGroup
from spacy.tokens import Span
```

O pilar de arrimo na infraestrutura para expandir a fronteira padronizada da engrenagem passa por encampar de antemão perante a via oficial (a rotina central descrita extensamente por aqui em meados da Parte II) um novo formato peculiar de cadastro atrelado na raiz do artefato central para **Span**. A premissa no caso se resume em carregar na mochila dessa entidade a chancela extra referenciando o tom “humorístico/mood”.

Já na esfera englobada operante na denominação particular em torno do utilitário abrigando viés de cunho **SpanGroup**, ele serve ao singelo fim propiciando agrupamentos customizados voltados a gerenciar caixas fechadas acopladas abrigando trechos de **Spans**, viabilizando ancorá-los na esteira do atributo pai de nome estipulado por **spans** nas entranhas da estrutura principal em *Doc*.

Logo, fabricaremos nosso conjunto em `SpanGroup`, englobando os segmentos providos originariamente a nível `sents` alocados no âmago interno.

A montagem base exigirá perante uso imperativo a determinação do modelo base a ser operado (em `doc`), estipulando chancelas em favor do selo norteador pelo quesito em `name`, por fim ancorando na lista em `spans` a leva e amostragem coligida do ninho mãe que acolheu as referidas sentenças!

```
# Cria um SpanGroup a partir dos Spans contidos no objeto Doc 'doc', que  
# foi criado a partir das anotações CoNLL-U. Esses Spans correspondem às  
# sentenças, cujos limites definimos manualmente. Atribui à variável  
# 'sent_group'.
```

```
sent_group = SpanGroup(doc=doc, name="sentences", spans=list(doc.sents))
```

Pronto. Deixamos as instâncias consolidadas a fim da inserção pontual, alocando a rubrica em favor de fixar raízes perante os arranjos no diretório em prol dos referidos sintagmas abrigados no domínio basilar no diretório central:

```
# Atribui o SpanGroup ao atributo 'spans' do objeto Doc, sob a chave  
# 'sentences'.
```

```
doc.spans['sentences'] = sent_group
```

Com o reduto operante garantido, fixamos o registro estendendo a alçada do novo selo “mood” (como propriedade de viés particular a ser fixado) sob as regras do arcabouço *Span*, inicializado de forma segura assumindo um valor cego padronizado de segurança estatuído nulo (`None`).

```
# Registra o atributo customizado 'mood' na classe Span
```

```
Span.set_extension('mood', default=None)
```

Nesse exato patamar é possível que passemos e apliquemos o fluxo rotineiro iterativo encarnando e transpondo engate simultâneo pautado em chaves pares casando “cada tom gramatical com sua oração nativa”. O Python cuida brilhantemente bem dessa proeza invocando sua magia incorporada de modo inerente em pauta através do operador unificador com viés emparelhador em “`zip()`”.

Lembrando, não é mágica — simplesmente vamos ditar a sintaxe base associando os componentes alinhados atrelados à alocada e nomeada `mood` amarrados por seu turno associando as frases batizadas provisoriamente sob alcunha da vertente em `span`.

Fincamos chancelas englobando um detalhe sutil e clássico do `spaCy` na transição de preenchimento balizando formatação na marca referenciada ao componente via notação singular em sublinhado “`_`”. Trata-se do reduto privativo reservado a inserções que partem pelo usuário (detalhe já repassado anteriormente!).

```
# Percorre os pares de itens da lista e do SpanGroup
```

```
for mood, span in zip(sent_types, doc.spans['sentences']):
```

```
# Atribui o valor de 'mood' ao atributo customizado de mesmo
```

```
# nome, pertencente ao objeto Span.
span._mood = mood
```

Agora nossos blocos textuais detêm uma carga informacional aprimorada!

Que tal pautar averiguação conferindo os arranjos nas chancelas amparadas na sentença alojada no endereço do marcador de número 8?

```
# Recupera a informação de modo verbal da sentença no índice 8
doc.spans['sentences'][8], doc.spans['sentences'][8]._mood
```

```
(If both your primary and reserve chutes have malfunctioned, signal immediately to a fellow
'imp')
```

E aí está. O modelo acusa na exatidão, e o diagnóstico assinala que a frase possui um modo “imperativo” enraizado em seu bojo gramatical. Funcionou como um relógio!

Tecendo o discurso com as anotações das relações RST

Chegou a hora e a vez de amarrarmos no pilar da ferramenta o estigma mais profundo da análise. O limite exato estipulando chancelas perante blocos de raciocínios independentes da teoria retórica se encontrava enlatado desde outrora nos arranjos guardados sob posse no acervo com estância referenciada nominalmente pela bagagem `discourse_units`.

```
# Verifica os 10 primeiros itens da lista 'discourse_units'
discourse_units[:10]
```

```
[0, 8, 11, 14, 19, 29, 34, 39, 51, 62]
```

Os indícios aqui apontam friamente onde uma “unidade retórica” brotava! E como transpor a estatística num formato orgânico? O viés lógico demanda pinçarmos recortes vivos no coração da estrutura original em *Doc* operando a faca exatamente por onde pautam as cercas balizadoras indicadas a nível de índices (ex: corta de zero a oito, de oito a onze, de onze a catorze, e sucessivamente até esvair o conteúdo)!

Esse desmembramento é facilmente vencido em pauta com o trato do encadeamento oriundo por via sequencial de loops operados em fôlego em `range()` fatiando pontualmente de praxe:

```
# Cria uma lista vazia para guardar as fatias do objeto Doc que correspondem
# às unidades de discurso.
edu_spans = []
```

```
# Percorre os limites das unidades de discurso usando a função range() do Python.
# Isso nos dá números, que usamos para indexar a lista 'discourse_units', a qual
# contém os índices que marcam o início de cada unidade de discurso.
for i in range(len(discourse_units)):
```

```

# Tenta executar o bloco de código a seguir
try:

    # Obtém o item atual da lista 'discourse_units' e o item seguinte; atribui
    # às variáveis 'start' e 'end'.
    start, end = discourse_units[i], discourse_units[i + 1]

    # Se o item seguinte não existir, porque chegamos ao último item da lista,
    # um IndexError será levantado, e nós o capturamos aqui.
except IndexError:

    # Atribui o início da unidade de discurso normalmente e usa o tamanho do
    # objeto Doc como valor de 'end', marcando o fim da unidade de discurso.
    start, end = discourse_units[i], len(doc)

    # Usa as variáveis 'start' e 'end' para fatiar o objeto Doc; acrescenta o
    # objeto Span resultante à lista 'edu_spans'.
    edu_spans.append(doc[start:end])

```

Com o esquadrinhamento selado na gaveta de repouso por trás das fronteiras contíguas moldadas na matriz listada `edu_spans`, temos a massa fundamental fragmentada.

```

# Obtém os sete primeiros Spans da lista 'edu_spans'
edu_spans[:7]

```

```

[How to Cope With a Double Parachute Failure,
While skydiving,,
it is possible,
(yet extremely unlikely),
that both your primary and reserve parachutes will malfunction,,
leaving you with no method,
of reducing your velocity.]

```

É latente a total indiferença entre o que se estipula chancelas num pilar da gramática ordinária para fixar sentenças longas em relação às métricas para moldar as chancelas da unidade discursiva, descompasso total!

Mas há peças cruciais pendentes focando os atributos necessários amarrando a premissa de relação e subordinação de um componente sob a mira do vizinho discursivo que moldará seu papel prático.

Logo criaremos mais três atributos sob estigma “feito em casa” no esqueleto em *Span*!

```

# Registra três atributos customizados para objetos Span, correspondentes ao
# id da unidade elementar de discurso, ao id do elemento que atua como alvo
# e ao nome da relação.
Span.set_extension('edu_id', default=None)

```



```
Span.set_extension('target_id', default=None)
Span.set_extension('relation', default=None)
```

Nesta engenharia: - **edu_id**: Grava o selo de identificação único para a porção elementar retórica. - **target_id**: Guarda quem é a outra porção conectada que sofre impacto prático. - **relation**: Rotula a natureza que define a ponte psicológica a entrelaçá-las.

E vamos novamente aninhar nossa ninhada fracionada de volta ao berço em *Doc* gerindo-as através da formatação de empacotamento com maestria englobada da já conhecida classe *SpanGroup*.

```
# Cria um objeto SpanGroup a partir dos Spans da lista 'edu_spans'
edu_group = SpanGroup(doc=doc, name="edus", spans=edu_spans)
```

```
# Atribui o SpanGroup sob a chave 'edus'
doc.spans['edus'] = edu_group
```

A etapa do arremate pede apenas que a gente alimente nossos “tubos de ensaio” vazios com o verdadeiro sangue englobado na matriz inicial (a tripla de tuplas guardada carinhosamente no banco com título de **relations**).

Ao puxarmos a lista originária veremos o amparo provido calcado em formatação atrelada a ditar “quem se comunica e como se comunica”.

```
# Imprime as cinco primeiras definições de relação da lista 'relations'
relations[:5]
```

```
[('preparation', 0, 10),
 ('circumstance', 1, 2),
 ('background', 2, 8),
 ('concession', 3, 2),
 ('same-unit', 4, 2)]
```

Resta a burocracia de encaminhar a transferência distribuindo a papelada para a respectiva subseção criada (**edus** no guarda-chuva de **spans**).

```
# Percorre cada tripla da lista 'relations'
```

```
for relation in relations:
```

```
# Desempacota a tripla em três variáveis
```

```
rel_name, source, target = relation[0], relation[1], relation[2]
```

```
# Usa o identificador em 'source' para indexar os objetos Span sob a
# chave 'edus'. Em seguida, acessa os atributos customizados e define os
# valores de 'edu_id', 'target_id' e 'relation'.
```

```
doc.spans['edus'][source]._.edu_id = source
doc.spans['edus'][source]._.target_id = target
doc.spans['edus'][source]._.relation = rel_name
```

Bingo! A trama retórica se encontra solidamente fixada ao osso da engrenagem!

Vamos averiguar o status retórico encravado no pedaço catalogado na posição sob índice de marcação de número 1:

```
# Recupera o atributo customizado 'relation' do Span no índice 1
doc.spans['edus'][1]._.relation

'circumstance'
```

Ele acusa operar na pauta “circunstancial” ditando o viés dependendo de amarras de praxe atreladas ao reduto apontando rumo na outra oração da extremidade espelhada balizada ao registro com “target_id”. Mas quem é o comparsa alvejado na mira do “target_id”?

```
# Obtém o Span no índice 1 e o Span referenciado por seu atributo 'target_id'
doc.spans['edus'][1], doc.spans['edus'][doc.spans['edus'][1]._.target_id]

(While skydiving,, it is possible)
```

Eis os gêmeos siameses desnudados: As porções encadeadas comprovando empiricamente e validando as amarras estipuladas entre as unidades elementares discursivas atreladas sob preceito referenciando chancelas do tipo que dita uma circunstância.

Atalhos para importar formatos CoNLL-U para dentro do spaCy

Caso não haja o apego emocional no ofício minucioso ao querer incorporar customizações artesanais para inflar a estância do modelo atrelada ao Doc e a intenção recaia friamente num interesse mecânico de fazer um passe mágico com viés restrito na pauta focada convertendo CoNLL-U num repositório puro formatado no padrão referenciando ao objeto balizado em spaCy, uma luz brilhante te aguarda: o método atalho sob o nome em utilitário de plantão chamado `conllu_to_docs()`.

Ele foi cunhado para viver no nicho englobando o amparo em setores base de processamentos para **training**, que figura na alçada balizando modelos alimentadores no treinamento focado sob as artérias de bases encorpadas pautando aprendizado computacional.

```
# Importa a função 'conllu_to_docs' e a classe Doc
from spacy.training.converters import conllu_to_docs
from spacy.tokens import Doc
```

Este artefato “mastiga” as entradas fornecidas via formato basilar operado na vertente **string**.

Empregamos no momento a massa bruta **annotations** fornecendo-a à máquina. E para que ela opere “quietinha”, escoramos o amparo chancelando premissas operantes na vertente que manda ela trancar a voz no quesito da opção atada e estatuída na marca **no_print** definida num “ok” positivo.

A rotina irá cuspir a produção (sob formato Pythonic em ‘generator’), de modo a requerer do usuário providências para converter tudo à moda clássica na moldura para base em formato “lista” (list).

```
# Fornece a string em 'annotations' à função 'conllu_to_docs'.  
# Define 'no_print' como True e converte o resultado em lista; armazena em 'docs'.  
docs = list(conllu_to_docs(annotations, no_print=True))
```

```
# Obtém os dois primeiros itens da lista resultante  
docs[:2]
```

[How to Cope With a Double Parachute Failure While skydiving, it is possible (yet extremely
Prepare for deployment. After linking arms with your fellow jumper, you will need to hook y

Pronto, criamos as fatias perfeitamente moldadas ao Doc. O pequeno tropeço na engrenagem que assombra essa facilidade balizada em padrão de fábrica: ela quebra arbitrariamente os documentos em maços picotados agrupando as ocorrências pautando-se sob limites focados ao amparo englobando 10 em 10 sentenças em caixinhas alocadas independentes de Docs individuais (um tanto desastroso).

Uma fusão coerente focada na reparação recai operando base numa vertente ditada a promover aglutinações embasadas a recompor o texto global num arcabouço atado na pauta unitária no viés em amarrações ancoradas nas chamadas à subrotina originada do artifício balizador estatuído sob a função `from_docs()` provida nativa na premissa com chancela perante molde embasando no coração *Doc*.

```
# Combina os objetos Doc da lista 'docs' em um único Doc; atribui a 'doc'  
doc = Doc.from_docs(docs)
```

```
# Verifica o tipo e o tamanho da variável  
type(doc), len(doc)
```

```
(spacy.tokens.doc.Doc, 890)
```

Milagre operado: Um belo e reluzente documento singular composto majestosamente em 890 matrizes providas de *Tokens* independentes! E é claro, com todas as virtudes de informações linguísticas incrustadas de nascença!

```
# Percorre os 8 primeiros Tokens usando a função range()  
for token_ix in range(0, 8):
```

```
# Usa o número atual em 'token_ix' para buscar um Token no Doc.  
# Atribui o objeto Token à variável 'token'.  
token = doc[token_ix]
```

```
# Imprime o Token e suas anotações linguísticas  
print(token, token.tag_, token.pos_, token.morph, token.dep_, token.head)
```

```
How WRB SCONJ PronType=Int mark Cope
to TO PART mark Cope
Cope VB VERB VerbForm=Inf ROOT Cope
With IN ADP case Failure
a DT DET Definite=Ind|PronType=Art det Failure
Double JJ ADJ Degree=Pos amod Failure
Parachute NN NOUN Number=Sing compound Failure
Failure NN NOUN Number=Sing obl Cope
```

Entretanto, não cante vitória de cara perante as virtudes. Ao esbarrar na tentação em requerer e focar em propriedades estatuindo sintagmas compostos enquadrando alocações na premissa originária de entidades em teor *noun_chunks*, a interface irá corar de vergonha na sua tela gritando e acusando por chancelas atadas em lacunas de dados que causarão a fúria em erros (“NotImplementedError”).

```
# Obtém os sintagmas nominais do Doc
list(doc.noun_chunks)
```

```
-----
NotImplementedError                                Traceback (most recent call last)
Cell In[47], line 2
      1 # Get the noun chunks in the Doc
----> 2 list(doc.noun_chunks)
```

```
File ~/nlp/lib/python3.9/site-packages/spacy/tokens/doc.py:853, in noun_chunks()
```

```
NotImplementedError: [E894] The 'noun_chunks' syntax iterator is not implemented for language
```

O revés ataca em formato oriundo puramente operando a nível estrito: o Doc gestado não porta qualquer cordão umbilical ligando a natureza a modelos estatuídos operantes do nível matricial de **Language** (*Vocabulary* e outros parâmetros que determinam o que a máquina sabe do idioma). E os extratos “noun chunks” em essência sugam de praxe, obrigatoriamente, embasamentos providos nestes dicionários linguísticos acoplados a suas raízes que chancelam sintaxe perante bases que o sistema ignora no instante atual.

Para pautar chancelas de praxe de cura e reabilitação não dispomos da possibilidade provida via injeção operando rotinas à força de maneira puramente e simplesmente ditar as cartilhas linguísticas à mão. O procedimento operante foca em balizas de “cirurgias” no armazenamento do formato “disco” acionando base da estância chamada de pilar *DocBin*, abordada em aprofundamento pretérito provido perante o horizonte da Parte II.

O pacote englobando as amarrações delineando no esteio pautado na pauta oriunda a estância abarcando propriedades base *DocBin* ostenta as prerrogativas chancelando enquadramento estipulando chancelas voltadas a transpor dados focados à exportação externa da estrutura pronta.

```
# Importa o objeto DocBin do submódulo 'tokens'
from spacy.tokens import DocBin
```

```
# Cria um objeto DocBin vazio
doc_bin = DocBin()
```

```
# Adiciona o Doc atual ao DocBin
doc_bin.add(doc)
```

Nesta pauta o revés reside ao invés do despejo em HD de arquivo bruto nós faremos o saque na fenda operando o recuo, onde engajaremos o saque imediato de volta para a pauta, puxando à luz do dia os extratos usando artifícios da técnica invocada em subrotina `get_docs()`, só que agora empurramos pela guelra da máquina o pacote que traz a carga focada em estatuir a língua que baliza o sistema de `Vocabulary`. O artefato que devolve vida inteligente perante amarrações amparadas e englobando o idioma no spaCy.

O fruto dessa manobra devolve uma safra “generator” e, logo, voltamos à lida transformando isso no velho enquadramento em lista (`list`).

```
# Usa o método 'get_docs' para recuperar os Docs do DocBin
docs = list(doc_bin.get_docs(vocab=nlp.vocab))
```

Problema sanado, o acesso volta à tona operando de maneira brilhante! A malha atada em viés referencial de propriedades contidas e embutidas sob chancela nos redutos amparando embates voltados ao uso da estrutura originária ditada por *noun chunks* se submete e jorra farta sem o engasgo da interdição perante a falta do idioma.

```
list(docs[0].noun_chunks)[:10]

[it,
 and reserve parachutes,
 this,
 these coping strategies,
 Steps,
 This,
 both your primary and reserve chutes,
 who,
 this,
 you]
```

Concluímos assim o mapeamento englobando o panorama referencial pautando-se sob chancelas a pavimentar vias da sua trilha em lidar e aprimorar a exploração das profundezas estruturais ancoradas no trato por anotações abrigadas perante o estigma ditado por regras estatuidas através da maestria provida nativa no formato operante sob regência do esquema abarcado por CoNLL-U operando a nível estrito focando suas intersecções atadas nas malhas do ecossistema e viés originário em componentes providos ao amparo de funcionalidades estipulando

enquadramento balizado na estância de spaCy.